ARTCRAFT FLEET TECHNICAL BROCHURE · SERIES #11

TravelCraft v0.2.0

TravelCraft is an ArtCraft-tier chrono-cartography studio and local-first travel guide platform that reinvents the classic 19th-century Baedeker guide for the 2030s.



v0.2.0	4 Crates	6771	6-Gate
RELEASE VERSION	WORKSPACE TIERS	PURE-RUST LOC	ARTCRAFT VERIFIED

Repository: `D:\Dev\repos\travelcraft` · **Standard:** `CRAFT_BROCHURE_STANDARD.md` · **Compiled via:** `BookCraft v0.5.0`

TravelCraft is an ArtCraft-tier chrono-cartography studio and local-first travel guide platform that reinvents the classic 19th-century Baedeker guide for the 2030s.

Features:

- **Chrono-Cartographic Layers**: Tokaido 1603 vs Neo-Tokyo; Hallstatt 800 BC vs Habsburg Vienna 1365.
- **Sensory Atmosphere Profiles**: Acoustic, architectural, olfactory, light records per waypoint.
- **Local Storage**: Offline SQLite journal + GPS-tagged photos via `DatabaseCraft`.
- **Stay-Length Itineraries**: 1/2/3/5/7/14-day Baedeker day-plans per edition (Itineraries tab, `travelcraft-cli itinerary AT 3`).
- **SightLens Camera**: Point laptop at Stephansdom and click Identify, or load an Immich export. Offline GPS + hint matching (Stephansdom, KHM Caravaggio Rosenkranzmadonna, Klimt Kiss, Bruegel Tower) renders a beautiful dossier and logs it with GPS source.
- **Sight Chat (text + voice)**: After identifying the Caravaggio, ask anything. Grounded in info blob + qgiscraft map blob + GTFS transit blob; online via Ollama `:11434` when reachable, else OFFLINE BLOB extractive answer. Voice out via Windows SAPI Speak or speech-mcp `:10909` TTS wav.
- **Maps + Transit blobs**: qgiscraft slippy tiles/scale/URLs (`:11254`) + preloaded Wiener Linien/JR snapshot; live departures via gtfs-mcp `:10913`.
- **High-Contrast Map**: Mouse-wheel zoom + drag pan, large type, ink/paper palette.
- **FastMCP Protocol**: Port `14129`.

Vendored offline sources (see `crates/travelcraft-baedeker/src/lib.rs SOURCES`):

- `japanophile-mcp/assets/knowledge/japan/travel.html` - JR Pass, konbini, budget, etiquette, regions.
- `vienna-life-assistant` `vienna_tips` (coffee/music/museum), `restaurants_data` Beisl rule, `culture_data` (KHM, Belvedere, Stephansplatz).

Who uses this, and why not just open Google Maps

Travelcraft is an offline, agent-native travel primer. The point is not a better map. It is that an agent in your IDE (Claude Desktop, Claude Code, Cursor, Zed, Antigravity, anything that speaks MCP) can plan, explain and keep a journal for a trip without a browser, an account, an API key or a network on the core path.

Who	Why not Google Maps	Typical prompt	Works today
Developer or power user with an agentic IDE	Stays in the tool, scriptable, no account, nothing leaves the machine	"Plan 3 days in Austria and put the stops in a table"	Yes, Austria and Japan only
Traveller with bad signal (tunnels, Tyrol, flights)	Core data is compiled in, no network needed	"What is the story of Stephansdom? I have no signal"	Yes, small catalogue
Culture and history traveller	Places are placed in time (chrono-waypoints), not just on a map	"What am I looking at at 48.2039, 16.3619?"	Yes, a handful of sights
Japan first-timer	Curated primer (JR Pass, konbini, etiquette) instead of ten browser tabs	"Plan a Tokaido route through Japan"	Yes, some text is German
Private journal keeper	Journal is a local SQLite file, survives crashes, never uploaded	"Log: Cafe Berg, rain, Melange, wet wool"	Yes
Builder of craft apps	Small reference for the ArtCraft pattern (engine, CLI, MCP, egui)	read `docs/PRD.md` and `AGENTS.md`	Yes

Prompts that work today

You say	Tool the agent calls	You get
"Which country editions are there?"	`travel_countries`	Austria and Japan editions
"Plan 3 days in Austria"	`travel_itinerary` (AT, 3)	Day-by-day plan (1, 2, 3, 5, 7 or 14 days)
"Best route through Japan, oldest sight first"	`travel_plan_route` (JP)	Chrono-ordered legs, distance, hours, carbon estimate
"What am I looking at near Stephansplatz?"	`travel_identify_sight`	Sight dossier from the offline catalogue
"How do I get to the Caravaggio in the KHM?"	`travel_sight_chat`	Grounded answer plus map and transit snapshot (not live departures)
"I photographed the Klimt Kiss, log it"	`travel_sightlens_capture`	Identification plus a durable journal entry
"Log: Cafe Berg, rain, Melange"	`travel_journal_add`	Saved entry id
"Show my last 10 journal entries"	`travel_journal_list`	Entries, newest first

You say	Tool the agent calls	You get
"Coordinates for Wien"	`travel_geocode`	Demo geocoder hit (a handful of places only)
"Vienna bicycle repair shops"	`travel_nearby` (bicycle_repair, Wien)	Nearest shops and repair stations from OpenStreetMap with address, phone, raw opening hours, OSM link. **Online**, cached 24 h
"Public toilets or drinking water near me"	`travel_nearby` (toilets, drinking_water, lat/lon)	Same, from OSM
"Which of these lack opening hours in OSM?"	`travel_nearby` `missing_tags` plus `osm_edit_url`	A to-do list for a mapper; travelcraft itself never edits OSM

Not built yet (do not expect these)

You say	Why it does not work today	What it would take
"Is it open now?", prices, booking	`travel_nearby` returns the raw `opening_hours` tag but nothing evaluates it, and there is no price or booking data	An opening-hours evaluator; booking is out of scope
"Another country"	Only Austria and Japan editions exist	New editions in `travelcraft-baedeker`
"Which painting is in this photo?"	Identification uses GPS plus a hint, not the image	An image model; not planned
Contributing back to OpenStreetMap	Travelcraft reads OSM tiles, it does not edit OSM	A mapper workflow belongs in giscraft, not here

Network use is limited to `travel_nearby` (and the GUI map tiles). Overpass data comes from public instances that are sometimes slow or down; travelcraft tries several mirrors, caches answers for 24 h and says so when it falls back to an old answer. Coverage and opening hours are only as good as OpenStreetMap.

Journal, chat history and settings live in `%LOCALAPPDATA%\travelcraft\travelcraft.db` (override with `TRAVELCRAFT_DATA`).

Workspace Structure

1. `crates/travelcraft-baedeker`: Editions (`JapanCraft` 9 stops, `AustriaCraft` 8 stops), stay itineraries, SightLens identify DB (landmarks + KHM/Belvedere artworks).

2. `crates/travelcraft-core`: Geodesic routing, itineraries, SightLens pipeline, durable SQLite journal (GPS columns), chat history and settings.

3. `crates/travelcraft-mcp`: MCP 2024-11-05 stdio server (13 tools, see `docs/TOOLS.md`). No port; hosts start the exe.

4. `crates/travelcraft-app`: Native GUI (`eframe`/`egui`) + CLI (`travelcraft-cli`).

Verification & Usage

```
# Run test suite
cargo test --workspace

# Headless smoke test (JP route + AT itinerary + Stephansdom + Caravaggio)
./target/debug/travelcraft.exe --smoke

# CLI
./target/debug/travelcraft-cli.exe plan JP
./target/debug/travelcraft-cli.exe itinerary AT 3
./target/debug/travelcraft-cli.exe identify 48.2085 16.3738 stephansdom
./target/debug/travelcraft-cli.exe identify 48.2039 16.3619 caravaggio
./target/debug/travelcraft-cli.exe capture immich-export.jpg --gps-source immich-exif --lat 48.2085 --lon 16.3738 --hint stephansdom
./target/debug/travelcraft-cli.exe chat --about caravaggio --question "Who painted the Rosenkranzmadonna and where is it?"
```

Sight Chat notes

- **Text**: Sight Chat tab, context pill shows current sight. Ask anything about the dossier; history (last 6) kept for follow-ups.
- **Online LLM**: Ollama `http://127.0.0.1:11434/api/chat` (model field, default `qwen2.5:7b`). System prompt = info blob + map + transit + sources. If unreachable, OFFLINE BLOB extractive answer is returned and labeled.
- **Voice out**: `Speak last answer` = Windows SAPI offline; `Save last answer (speech-mcp wav)` = POST `:10909/api/v1/tts` to temp wav.
- **Voice in**: type/paste transcript (mic path: speech-mcp `:10909` web UI + FunASR sidecar `:10914`; record wav then paste transcript here in v1).
- **Maps**: every prompt carries `MAP qgiscraft Z16 tile + OSM URL + scale + Z8 bbox count` (offline math, live tiles via qgiscraft `:11254`).
- **Transit**: preloaded blob per sight (Stephansdom U1/U3, KHM D/1/2/71 + U3, Belvedere D/71, Schoenbrunn U4, JP JR/IC hubs) sourced from `gtfs-mcp/data/vienna|wien|default`; live departures via gtfs-mcp `:10913`.

SightLens GPS notes

- **Windows location**: Settings > Privacy > Location ON; browser/OS prompt once.
- **iPhone as hotspot**: shares internet only, NOT GPS. Enter coords manually or import the Immich original (EXIF GPS preserved).
- **Immich**: export original (not preview), paste path, source `immich-exif`.

Status - does today / next week / non-goals

Does today: 4-crate Rust workspace (baedeker/core/mcp/app); Baedeker guide model (places, routes, itineraries); map app with fly-to, scale bar, regional intel; fleet i18n (7 locales); 11 test fns; CLI verbs via `travelcraft --help`.

Next week: `docs/PRD.md` (model + verbs + qgiscraft boundary) plus `docs/TOOLS.md`; `just ci` (fmt + clippy-deny + test); first democraft recipe on fixtures.

Non-goals: not a GIS analyzer (that's qgiscraft); no live-tile SLA - keyless OSM/Photon only.

Lineage - the Storytold craft-app pattern

Storytold ArtCraft (storytold/artcraft - "the IDE for artists") was the starting point. The 16 craft apps in the Storytold stable each ship one binary that is both the human CLI and the agent MCP server (stdio), headless-first, with `run.json` + report artifacts. That modus operandi and harness is what this repo implements; the code here is independent, the pattern is shared.

Product Requirements & Architecture Specification

What it is

A local-first, offline travel primer for Austria and Japan in the Baedeker spirit: places placed in time (chrono-waypoints), what they feel like (sensory atmosphere), how to route between them, and what to know on the ground. Pure Rust, ArtCraft pattern: one engine, three front ends (CLI, MCP stdio, egui app).

Users

Sandra and her agents. Agents use the MCP tools to plan stays and answer "what is this" questions. Sandra uses the egui app and the iPad dossier page.

Today

- Editions: Japan (9 stops), Austria (8 stops), each with a knowledge tree.
- Planning: haversine route optimiser, stay itineraries (1, 2, 3, 5, 7, 14 days).
- SightLens: identify a sight from GPS plus a hint, log a photo to the durable

journal (SQLite, WAL). Sight Chat: grounded offline answer, optional Ollama in the GUI.

- Surfaces: `travelcraft` (egui), `travelcraft-cli`, `travelcraft-mcp` (13 tools,

see TOOLS.md), `travelcraft-cli serve` (iPad dossier page), `publish` to the

fleet Intel Hub.

Boundaries

- Not a GIS analyzer. Map maths and the demo geocoder come from giscraft-core.
- No live-tile SLA. Keyless OSM and Photon only.
- No account, no cloud dependency for the core path. Offline is the thesis.
- giscraft dependency: path dependency on giscraft-core for map maths (`geo::haversine_m`, `LonLat`) and the demo geocoder behind `travel_geocode` only. giscraft's Phase 0 (its `docs/PHASE0-foundations.md`, 2026-11-08, designed in another session and not merged) is additive and changes none of those, so nothing here needs to change. Travelcraft keeps its own Overpass client and cache on purpose: giscraft's overpass module is moving to attribute-preserving hits and a command registry. Revisit after that merges; `OverpassHit.props` could then replace the tag parsing in `nearby.rs`. This repo never edits giscraft.

Gaps (honest)

- Catalogue is tiny; identify matches by hint and distance, not by image.
- Overpass mirror reliability is undiagnosed: from this machine overpass-api.de answers 502 or hangs, kumi.systems and private.coffee hang, while overpass.openstreetmap.fr and overpass.osm.ch answer in about a second. Cause unknown (rate limiting of this IP, routing, or the servers). Needs a proper look with timings per mirror over a day. Until then the French mirror goes first.
- POI search works only through `travel_nearby` (online, public Overpass mirrors that are sometimes down). Place names resolve only for ten AT and JP cities; everything else needs lat and lon. No opening-hours evaluation.
- Live transit (gtfs-mcp) and voice paths are GUI-side and need other services.
- Offline region pack format is not designed yet.
- egui app crate still uses unwrap and expect and is outside the deny-lints.

Next (days, not weeks)

1. Grow the catalogue and add a catalogue loader instead of compiled-in data.
2. Bring the egui crate under the deny-lints.
3. NSIS current-user installer and an mcpb bundle, per the fleet standard.

Complete MCP Tool & CLI Verb Reference

Server: `travelcraft-mcp` (MCP 2024-11-05, line-delimited JSON-RPC on stdio).

stdout carries protocol only, logs go to stderr. Notifications get no reply.

Errors: `{success:false, error, error_type}` with `isError: true`.

State is durable: journal, chat history and settings live in a SQLite file (WAL, synchronous=FULL),

`%LOCALAPPDATA%\travelcraft\travelcraft.db` or `$TRAVELCRAFT_DATA\travelcraft.db`. A crash loses nothing

that was acknowledged. If the database cannot be opened the server exits with an error; it does not

fall back to memory.

Tool	Arguments	Returns
`travel_status`	none	editions, persistence flag, db path, journal count, vendored source list
`travel_countries`	none	all country editions (large, about 14 KB)
`travel_inspect`	`country` (AT, JP)	one edition: waypoints, atmosphere, curated routes
`travel_knowledge_tree`	`country`	primer topic tree (large, about 18 KB for JP)
`travel_plan_route`	`country`	chrono-optimised route: legs, distance, hours, carbon, map tiles
`travel_itinerary`	`country`, `days` (1, 2, 3, 5, 7, 14; default 3)	day-by-day plan
`travel_identify_sight`	`lat`, `lon`, `hint` (all optional)	sight or artwork from the offline catalogue
`travel_sight_chat`	`question`; optional `about`, `lat`, `lon`	offline answer plus system prompt, map blob, transit blob
`travel_sightlens_capture`	`photo`; optional `gps_source`, `lat`, `lon`, `hint`	identification plus a durable journal entry
`travel_journal_add`	`reflection`; optional `sensory_notes`, `waypoint_id`, `photo`, `lat`, `lon`, `gps_source`	new entry id; text capped at 16000 chars
`travel_journal_list`	optional `limit` (1 to 1000, default 20)	entries, newest first
`travel_geocode`	`query`	offline demo geocoder hit (giscraft-core); echoes the query at a fixed Wien point, not a place index

Tool	Arguments	Returns
`travel_nearby`	`category` or `key` + `value`; `lat` + `lon` or `place` (major AT/JP city); optional `radius_m` (50 to 5000, default 1500), `limit` (1 to 50, default 15), `refresh`	**online, opt-in.** Places from OpenStreetMap via Overpass, nearest first, with OSM and edit links, raw opening_hours, `missing_tags`, ODbL attribution, and `source` (overpass, cache, stale_cache)

Network: `travel_nearby` is the only tool that uses it. Categories: bicycle_repair, bicycle_rental, pharmacy, doctors, hospital, atm, cafe, restaurant, bakery, supermarket, toilets, drinking_water, ev_charging, post_office, museum, viewpoint, hotel, train_station; or any fenced OSM key and value. No raw Overpass QL is accepted. OSM text is untrusted data: control characters are stripped and fields capped at 200 chars. Answers are cached 24 h in the database; if the live query fails, the last cached answer comes back with `stale: true` and a warning, and with no cache the call fails with `error_type: offline`. Public mirrors are tried in order (overpass.openstreetmap.fr, overpass-api.de, kumi.systems, private.coffee); `TRAVELCRAFT_OVERPASS_URL` pins one endpoint. `truncated: true` means Overpass hit the 300-element fetch cap, so shrink the radius. Opening hours are returned raw and never evaluated.

Limits today: the catalogue is small (Stephansdom, KHM Caravaggio, Klimt Kiss and the edition waypoints). `travel_sight_chat` does not call an LLM; it returns the grounding for the caller's own model. Live transit and live map tiles are not fetched by the MCP server.

Wire it in a host (Claude Desktop shape):

```
{ "mcpServers": { "travelcraft": { "command":
"D:\\Dev\\repos\\travelcraft\\target\\release\\travelcraft-mcp.exe" } } }
```